Actividad 3 Optimización Matemática

Juan Camilo Tobar Valencia

Roger Rafael Lopesierra Olivera

Santiago García Restrepo

Jesús Andrés Jiménez Pachón

Docente

Whady Felipe Flórez Escobar

Universidad Pontificia Bolivariana

Maestría en Ciencias Naturales y Matemáticas

2026

Ejercicio 1 — Efecto de `step_size` y T en Basin Hopping

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: Modifique `scripts/01_basin_hopping.py` probando `step_size` $\in \{2, 5, 10, 15\}$ y $T \in \{0{,}3, 1{,}5, 5\}$. ¿Qué combinación encuentra el global con más frecuencia en 10 corridas con semillas distintas?

1. Definiciones necesarias

1.1. Algoritmo de basin hopping (recordatorio)

1. Partir de un mínimo local actual x_{cur} (vía optimizador local).
2. Perturbar: $x_{\text{prop}} = x_{\text{cur}} + \Delta$, con Δ un salto aleatorio de magnitud `step_size`.
3. Reoptimizar localmente desde $x_{\text{prop}} \to x_{\text{new}}, f_{\text{new}}$.
4. Aceptar con criterio de Metrópolis: siempre si $f_{\text{new}} < f_{\text{cur}}$; si no, con probabilidad $P = e^{-\Delta E/T}$.
5. Guardar aparte el mejor $(x_{\text{best}}, f_{\text{best}})$ visto en toda la corrida.

1.2. El compromiso del `step_size`

- Muy chico: el salto casi nunca saca al punto de la cuenca actual → se queda atascado.
- Muy grande: cae en zonas lejanas al azar → degenera a random restart.
- Razonable: comparable a la distancia típica entre mínimos locales del paisaje (aquí, $\sim$ 9–13 unidades).

1.3. El efecto de la temperatura T

- T baja: casi nunca acepta empeorar (como random walk greedy).
- T alta: acepta empeorar casi siempre (como random walk puro).
- T intermedia: balance entre explotar el mínimo actual y explorar otras cuencas.

2. Adaptación del código (por eficiencia computacional)

Se usó el mismo paisaje de 6 pozos y la misma lógica de Metrópolis de `01_basin_hopping.py`, pero se reemplazó el optimizador local de descenso de gradiente manual (paso fijo, hasta 2000 iteraciones) por **BFGS de scipy** como `local_search`, ya que el barrido de 12 combinaciones $\times$ 10 semillas $\times$ 40 saltos = 4,800 reoptimizaciones locales sería demasiado lento con gradiente puro. Esta sustitución es exactamente la misma que usa el script original para la comparación "manual vs. scipy", por lo que no cambia la esencia del algoritmo.

```python
import numpy as np
from scipy.optimize import minimize

POZOS = [
    (-5.0, -6.5, 4.0, 1.3),
    (-4.0,  5.0, 4.0, 1.1),
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

def local_search_bfgs(x0):
    res = minimize(lambda p: f(p[0], p[1]), x0, method="BFGS")
    return res.x, float(res.fun)

def basin_hopping_manual_fast(x0, n_iter=40, step_size=7.0, T=1.5, seed=0)
    :
    rng = np.random.default_rng(seed)
    x_cur, f_cur = local_search_bfgs(x0)
    x_best, f_best = np.array(x_cur), f_cur
    for it in range(1, n_iter + 1):
        x_prop = x_cur + rng.uniform(-step_size, step_size, size=2)
        x_new, f_new = local_search_bfgs(x_prop)
        if f_new < f_cur:
            aceptado = True
        else:
            aceptado = rng.random() < np.exp(-(f_new - f_cur) / T)
        if aceptado:
            x_cur, f_cur = x_new, f_new
            if f_new < f_best:
                x_best, f_best = np.array(x_new), f_new
    return np.array(x_best), f_best
```

Verificación: la implementación reproduce el resultado de la Clase 3

```python
x0 = np.array([8.0, -8.0])
xb, fb = basin_hopping_manual_fast(x0, n_iter=40, step_size=7.0, T=1.5,
    seed=0)
print(xb, fb)
```

Resultado: $x^* = (-6{,}3698,\ 3{,}9191)$, $f^* = -3{,}28800$ — coincide exactamente con $x^* \approx (-6{,}37, 3{,}92)$, $f^* \approx -3{,}288$ reportado en la Clase 3.

3. Código del barrido de experimentos

```
f_global = -3.2879983559811907
tol_exito = 0.01  # se considera "exito" si f_best esta a menos de 0.01 del global

step_sizes = [2, 5, 10, 15]
temperaturas = [0.3, 1.5, 5]
seeds = list(range(10))

resultados = {}
for step in step_sizes:
    for T in temperaturas:
        exitos = 0
        f_vals = []
        for seed in seeds:
            xb, fb = basin_hopping_manual_fast(x0, n_iter=40, step_size=step, T=T, seed=seed)
            f_vals.append(fb)
            if abs(fb - f_global) < tol_exito:
                exitos += 1
        resultados[(step, T)] = (exitos, np.mean(f_vals), np.min(f_vals), np.max(f_vals))
```

4. Resultados obtenidos (10 corridas por combinación)

Código para generar la tabla

```
print("### 4. Resultados obtenidos (10 corridas por combinacion)\n")

# Encabezado
header = f"{'step_size':>10} | {'T':>5} | {'Exitos / 10':>11} | {'f promedio':>12} | {'f minimo':>10} | {'f maximo':>10}"
print(header)
print("-" * len(header))

# Filas de datos
for (step, T), (exitos, f_mean, f_min, f_max) in resultados.items():
    print(f"{step:10d} | {T:5.1f} | {exitos:11d} | {f_mean:12.4f} | {f_min:10.4f} | {f_max:10.4f}")
```

Salida obtenida

```
step_size |     T | Exitos / 10 |   f promedio |   f minimo |   f maximo
-------------------------------------------------------------------------
         2 |   0.3 |           0 |      -0.0000 |    -0.0000 |    -0.0000
         2 |   1.5 |           0 |      -0.0000 |    -0.0000 |    -0.0000
         2 |   5.0 |           0 |      -0.0000 |    -0.0000 |    -0.0000
         5 |   0.3 |           2 |      -2.6927 |    -3.2880 |    -1.7996
         5 |   1.5 |           8 |      -3.1888 |    -3.2880 |    -2.7920
         5 |   5.0 |          10 |      -3.2880 |    -3.2880 |    -3.2880
```

```
        10 |    0.3 |           7 |       -3.1392 |     -3.2880 |     -2.7920
        10 |    1.5 |          10 |       -3.2880 |     -3.2880 |     -3.2880
        10 |    5.0 |          10 |       -3.2880 |     -3.2880 |     -3.2880
        15 |    0.3 |           6 |       -2.7656 |     -3.2880 |     -0.5442
        15 |    1.5 |           7 |       -3.1392 |     -3.2880 |     -2.7920
        15 |    5.0 |           7 |       -3.1392 |     -3.2880 |     -2.7920
```

Tabla equivalente en formato de documento

`step_size`	T	Éxitos / 10	f promedio	f mínimo	f máximo
2	0.3	0	−0,0000	−0,0000	−0,0000
2	1.5	0	−0,0000	−0,0000	−0,0000
2	5.0	0	−0,0000	−0,0000	−0,0000
5	0.3	2	−2,6927	−3,2880	−1,7996
5	1.5	8	−3,1888	−3,2880	−2,7920
5	5.0	**10**	−3,2880	−3,2880	−3,2880
10	0.3	7	−3,1392	−3,2880	−2,7920
10	1.5	**10**	−3,2880	−3,2880	−3,2880
10	5.0	**10**	−3,2880	−3,2880	−3,2880
15	0.3	6	−2,7656	−3,2880	−0,5442
15	1.5	7	−3,1392	−3,2880	−2,7920
15	5.0	7	−3,1392	−3,2880	−2,7920

Cuadro 1: Éxitos y estadísticas de f^* por cada combinación de `step_size` y T (10 semillas).

Las combinaciones con 10/10 éxitos son: $(step = 5,\ T = 5)$, $(step = 10,\ T = 1{,}5)$, $(step = 10,\ T = 5)$.

5. Análisis por cada parámetro

5.1. Efecto del `step_size`

- `step_size=2` **(demasiado chico):** falla en el 100 % de los casos, sin excepción. El salto es tan pequeño que casi siempre reoptimiza de vuelta al mismo pozo del que partió, quedando atrapado en $f \approx 0$ — exactamente el "falso mínimo" entre pozos que la Clase 3 menciona como trampa de Powell. Esto confirma directamente la predicción teórica: *"step_size muy chico: el salto casi nunca saca al punto de la cuenca actual"*.

- `step_size=5`**:** funciona bien solo con temperatura alta ($T = 5$, 10/10 éxitos), pero falla mucho con temperatura baja ($T = 0{,}3$, solo 2/10).

- `step_size=10`**:** es el más robusto en general — logra 10/10 con $T = 1{,}5$ y $T = 5$, y todavía 7/10 con $T = 0{,}3$. Esto es consistente con el valor `step_size=7.0` usado por defecto en el script de la Clase 3 (comparable a la separación entre pozos, $\sim$ 9–13 unidades).

- `step_size=15` **(demasiado grande):** el desempeño **empeora** respecto a `step=10` en todas las temperaturas (6-7 éxitos en vez de 7-10). Esto confirma la otra mitad de la predicción teórica:

"step_size muy grande: el salto cae en zonas lejanas y casi al azar del paisaje", perdiendo la ventaja de explorar cuencas vecinas de forma dirigida.

5.2. Efecto de la temperatura T

- $T = 0{,}3$ **(fría):** en general reduce el número de éxitos porque casi nunca acepta un salto peor, quedando fácilmente atrapado en el primer mínimo local razonable que encuentra (comportamiento cercano a random walk greedy).
- $T = 1{,}5$ **y** $T = 5$ **(templada/caliente):** consistentemente mejores, porque permiten aceptar saltos temporalmente peores, lo que es exactamente lo que posibilitó el escape hacia el pozo global en el ejemplo paso a paso de la Clase 3 (iteración 6, "ACEPTADO (peor, sorteo!)").

6. Combinación recomendada

Mejor combinación: $step_size = 10,\ T = 1{,}5$ (o $T = 5$), ambas con 10/10 éxitos.

Se recomienda $step_size = 10$, $T = 1{,}5$ sobre $T = 5$ porque una temperatura más baja, cuando ya es suficiente para escapar, reduce el riesgo de "deshacer" un buen progreso aceptando saltos malos innecesarios en corridas más largas — aunque en este experimento de 40 iteraciones ambas temperaturas fueron igualmente efectivas.

7. Interpretación general

Este experimento confirma cuantitativamente el **compromiso** teórico descrito en la Clase 3 para ambos parámetros:

- El `step_size` tiene un **punto óptimo intermedio** (aquí, ≈ 10): ni tan chico que no escape de la cuenca actual, ni tan grande que pierda la estructura del paisaje.
- La temperatura T funciona mejor cuando es **suficientemente alta** para permitir escapes, confirmando que el criterio de Metrópolis con temperatura fija es sensible a esta elección, y motivando el uso de temperaturas **decrecientes** (simulated annealing, siguiente sección de la Clase 3) para obtener lo mejor de ambos mundos: exploración al inicio, explotación al final.

Conclusión final: ninguna combinación de parámetros de basin hopping garantiza el 100 % de éxito salvo aquellas donde tanto el salto como la temperatura son "generosos" mas no excesivos; esto ilustra por qué basin hopping "mejora las probabilidades de encontrar el global, no las garantiza en una sola corrida", tal como advierte explícitamente el material de la Clase 3.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.

Ejercicio 2 — Enfriamiento Lineal vs. Geométrico en Simulated Annealing

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: Implemente un esquema de enfriamiento lineal ($T_k = T_0(1 - k/n_{\text{iter}})$) para simulated annealing y compárelo contra el geométrico del script `04_simulated_annealing.py`.

1. Definiciones necesarias

1.1. Simulated annealing (recordatorio)

Diferencia clave con basin hopping: no reoptimiza localmente en cada paso. Perturba el punto actual y compara/acepta **directamente** con el criterio de Metrópolis; el "aterrizaje" en el mínimo ocurre solo al final, cuando T ya es muy chica.

1.2. Esquema de enfriamiento geométrico (el usado en el script original)

$$T_k = T_0\,\alpha^k, \qquad 0 < \alpha < 1 \text{ (típicamente 0.90–0.99).}$$

Este esquema decae **exponencialmente**: pierde una fracción constante $(1-\alpha)$ de temperatura en cada paso.

1.3. Esquema de enfriamiento lineal (a implementar)

$$T_k = T_0\left(1 - \frac{k}{n_{\text{iter}}}\right).$$

Este esquema decae **linealmente**: pierde una cantidad absoluta constante T_0/n_{iter} de temperatura en cada paso, llegando a $T \approx 0$ exactamente en la última iteración.

2. Código: ambos esquemas de enfriamiento

```python
import numpy as np

POZOS = [
    (-5.0, -6.5, 4.0, 1.3),
    (-4.0,  5.0, 4.0, 1.1),
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]
```

```python

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

def simulated_annealing_geometrico(x0, T0=8.0, alpha=0.96, n_iter=200,
    paso=1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur = np.array(x0, dtype=float)
    f_cur = float(f(*x_cur))
    x_best, f_best = x_cur.copy(), f_cur
    historia, T_historia = [x_cur.copy()], []
    T = T0
    for it in range(n_iter):
        x_prop = x_cur + rng.normal(scale=paso, size=2)
        f_prop = float(f(*x_prop))
        dE = f_prop - f_cur
        aceptado = dE < 0 or rng.random() < np.exp(-dE / T)
        if aceptado:
            x_cur, f_cur = x_prop, f_prop
            if f_cur < f_best:
                x_best, f_best = x_cur.copy(), f_cur
        historia.append(x_cur.copy())
        T_historia.append(T)
        T *= alpha            # <-- ENFRIAMIENTO GEOMETRICO
    return x_best, f_best, np.array(historia), np.array(T_historia)


def simulated_annealing_lineal(x0, T0=8.0, n_iter=200, paso=1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur = np.array(x0, dtype=float)
    f_cur = float(f(*x_cur))
    x_best, f_best = x_cur.copy(), f_cur
    historia, T_historia = [x_cur.copy()], []
    for k in range(n_iter):
        T = T0 * (1 - k / n_iter)     # <-- ENFRIAMIENTO LINEAL
        T = max(T, 1e-8)              # evita division por 0 en la ultima
            iteracion
        x_prop = x_cur + rng.normal(scale=paso, size=2)
        f_prop = float(f(*x_prop))
        dE = f_prop - f_cur
        aceptado = dE < 0 or rng.random() < np.exp(-dE / T)
        if aceptado:
            x_cur, f_cur = x_prop, f_prop
            if f_cur < f_best:
                x_best, f_best = x_cur.copy(), f_cur
        historia.append(x_cur.copy())
        T_historia.append(T)
    return x_best, f_best, np.array(historia), np.array(T_historia)
```

3. Comparación con la misma semilla del script original (seed=0)

```
x0 = np.array([8.0, -8.0])

xb_geo, fb_geo, hist_geo, T_geo = simulated_annealing_geometrico(x0, T0
    =8.0, alpha=0.96, n_iter=200, paso=1.5, seed=0)
xb_lin, fb_lin, hist_lin, T_lin = simulated_annealing_lineal(x0, T0=8.0,
    n_iter=200, paso=1.5, seed=0)

print(f"Geometrico -> x* = {xb_geo.round(4)} f* = {fb_geo:.4f}    T: {
    T_geo[0]:.1f} -> {T_geo[-1]:.5f}")
print(f"Lineal     -> x* = {xb_lin.round(4)} f* = {fb_lin:.4f}    T: {
    T_lin[0]:.1f} -> {T_lin[-1]:.5f}")
```

Resultado obtenido

```
Geometrico -> x* = [-6.3514  4.0089] f* = -3.2754    T: 8.0 -> 0.00237
Lineal     -> x* = [-6.2745  3.5959] f* = -3.1263    T: 8.0 -> 0.04000
```

Con la misma semilla, ambos esquemas encuentran el pozo **global** (cerca de $x^* \approx (-6{,}37, 3{,}92)$), pero el enfriamiento **geométrico** aterriza más cerca del fondo exacto ($f^* = -3{,}2754$) que el **lineal** ($f^* = -3{,}1263$), porque termina con una temperatura final ~ 17 veces menor: $T_{\text{geo,final}} = 0{,}00237$ vs. $T_{\text{lin,final}} = 0{,}04000$.

4. Comparación de las curvas de temperatura

```
T0, alpha, n_iter = 8.0, 0.96, 200
Tg = T0 * alpha ** np.arange(n_iter)
Tl = T0 * (1 - np.arange(n_iter) / n_iter)

metricas = [
    ("T promedio en las primeras 50 iteraciones", Tg[:50].mean(), Tl[:50].
        mean()),
    ("T promedio en las ultimas 50 iteraciones", Tg[-50:].mean(), Tl
        [-50:].mean()),
    ("T en la iteracion 100 (mitad del proceso)", Tg[100], Tl[100]),
    ("T final (iteracion 200)", Tg[-1], Tl[-1]),
    ("Movimientos aceptados (de 200)", acc_g, acc_l),
]

header = f"{'Metrica':45} | {'Geometrico':>12} | {'Lineal':>10}"
print(header)
print("-" * len(header))
for nombre, vg, vl in metricas:
    print(f"{nombre:45} | {vg:12.4f} | {vl:10.4f}")
```

Resultado obtenido

```
Metrica                                                |   Geometrico |
   Lineal
-------------------------------------------------------------------------------

T promedio en las primeras 50 iteraciones              |       3.4805 |
   7.0200
T promedio en las ultimas 50 iteraciones               |       0.0076 |
   1.0200
T en la iteracion 100 (mitad del proceso)              |       0.1350 |
   4.0000
T final (iteracion 200)                                |       0.0024 |
   0.0400
Movimientos aceptados (de 200)                         |      69.0000 |
   143.0000
```

Tabla equivalente en formato de documento

Métrica	Geométrico ($\alpha = 0{,}96$)	Lineal
T promedio en las primeras 50 iteraciones	3,48	7,02
T promedio en las últimas 50 iteraciones	0,0076	1,02
T en la iteración 100 (mitad del proceso)	0,135	4,00
T_{final} (iteración 200)	0,00237	0,04
Movimientos aceptados (de 200)	69	143

Cuadro 1: Comparación de las curvas de temperatura entre el esquema geométrico y el lineal (semilla 0).

El enfriamiento lineal permanece çaliente"mucho más tiempo que el geométrico.

Esta es la diferencia estructural clave: en la iteración 100 (justo la mitad del proceso), el esquema geométrico ya está en $T \approx 0{,}135$ (prácticamente en fase de explotación pura), mientras que el lineal todavía está en $T = 4{,}0$ (bastante "caliente", en plena fase exploratoria). Por eso el esquema lineal acepta **más del doble** de movimientos (143 vs. 69), reflejando que sigue aceptando saltos peores mucho más tarde en el proceso.

5. Comparación de robustez con 10 semillas distintas

```
f_global = -3.2879983559811907
tol_exito = 0.05

seeds = list(range(10))
resultados_geo, resultados_lin = [], []
for seed in seeds:
    _, fb_g, _, _ = simulated_annealing_geometrico(x0, T0=8.0, alpha=0.96,
        n_iter=200, paso=1.5, seed=seed)
    _, fb_l, _, _ = simulated_annealing_lineal(x0, T0=8.0, n_iter=200,
       paso=1.5, seed=seed)
```

```
    resultados_geo.append(fb_g)
    resultados_lin.append(fb_l)

resultados_geo = np.array(resultados_geo)
resultados_lin = np.array(resultados_lin)

exitos_geo = np.sum(np.abs(resultados_geo - f_global) < tol_exito)
exitos_lin = np.sum(np.abs(resultados_lin - f_global) < tol_exito)

header = f"{'Metodo':12} | {'Exitos/10':>9} | {'f promedio':>11} | {'f minimo':>10} | {'f maximo':>10}"
print(header)
print("-" * len(header))
print(f"{'Geometrico':12} | {exitos_geo:9d} | {resultados_geo.mean():11.4f} | {resultados_geo.min():10.4f} | {resultados_geo.max():10.4f}")
print(f"{'Lineal':12} | {exitos_lin:9d} | {resultados_lin.mean():11.4f} | {resultados_lin.min():10.4f} | {resultados_lin.max():10.4f}")
```

Resultado obtenido

```
Metodo       | Exitos/10 |  f promedio |   f minimo |   f maximo
-----------------------------------------------------------------
Geometrico   |         2 |     -2.5258 |    -3.2754 |    -0.4998
Lineal       |         0 |     -2.6734 |    -3.2028 |    -0.9811
```

Tabla equivalente en formato de documento

Método	Éxitos exactos (tol 0,05)	f promedio	f mínimo	f máximo
Geométrico	2/10	−2,5258	−3,2754	−0,4998
Lineal	0/10	−2,6734	−3,2028	−0,9811

Cuadro 2: Robustez de cada esquema de enfriamiento sobre 10 semillas distintas ($seed = 0, \ldots, 9$).

6. Interpretación de los resultados

¿Por qué el geométrico tiende a dar valores finales más precisos?

Aunque ninguno de los dos esquemas alcanza el mínimo exacto con alta frecuencia en solo 200 iteraciones sin reoptimización local (a diferencia de basin hopping, que sí reoptimiza), el esquema **geométrico** tiende a producir valores de f^* ligeramente mejores en promedio, porque:

- **Enfría más agresivamente al inicio:** después de solo 30–40 iteraciones ya reduce T sustancialmente, concentrando el "presupuesto de exploración" en las primeras iteraciones, cuando el punto está lejos y necesita moverse mucho.
- **Permite una fase de explotación fina más larga:** con T muy baja durante la segunda mitad del proceso, el punto se asienta suavemente en el fondo del pozo que encontró, refinando la posición.

¿Por qué el lineal es menos preciso aquí, con los mismos parámetros?

El esquema lineal, con $T_0 = 8$ y $n_{\text{iter}} = 200$, mantiene una temperatura relativamente alta durante **más de la mitad** del proceso (en la iteración 100, todavía tiene $T = 4{,}0$, la mitad de la inicial). Esto significa que sigue "caminando" con bastante libertad incluso cuando ya debería estar refinando su posición, sin suficiente tiempo al final para asentarse con precisión en el fondo exacto del pozo.

Esto no es una limitación inherente del enfriamiento lineal

Es importante notar que esta comparación usa el **mismo** $T_0 = 8{,}0$ para ambos esquemas, lo cual no es una comparación "justa" en términos de área bajo la curva de temperatura. El enfriamiento lineal podría mejorarse ajustando T_0 a un valor menor, de modo que termine con una temperatura final comparable a la del esquema geométrico, o alargando n_{iter} para dar más tiempo a la fase de explotación.

7. Resultado final

Geometrico ($seed = 0$): $f^* = -3{,}2754$, Lineal ($seed = 0$): $f^* = -3{,}1263$

Robustez (10 semillas): geometrico 2/10 exitos exactos vs. lineal 0/10, con promedios similares ($\sim -2{,}5$ a $-2{,}7$

Conclusión: el esquema de enfriamiento geométrico del script original ($T_k = T_0\alpha^k$) tiende a ser ligeramente más preciso que un esquema lineal ingenuo con los mismos parámetros (T_0, n_{iter}), porque distribuye el "tiempo caliente" de forma más eficiente: mucha exploración muy al principio, seguida de una fase de explotación fina proporcionalmente más larga. El esquema lineal, al perder temperatura de forma más "pareja", pasa más tiempo en un régimen intermedio (ni completamente explorando ni completamente explotando), lo que en este paisaje de 6 pozos se traduce en una convergencia final ligeramente menos precisa, aunque ambos esquemas logran identificar la región del mínimo global en la mayoría de los casos.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.

Ejercicio 3 — Efecto del Elitismo en el Algoritmo Genético

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: En el algoritmo genético, quite el elitismo ($n_{\text{elite}} = 0$) y observe qué pasa con la curva de convergencia. ¿Por qué puede el mejor f empeorar de una generación a otra sin elitismo?

1. Definiciones necesarias

1.1. Algoritmo genético (recordatorio)

En vez de mover un solo punto, se mantienen N soluciones candidatas (población) que evolucionan mediante:

1. **Fitness:** qué tan buena es cada solución ($-f$, porque minimizamos f y "fitness alto = mejor").
2. **Selección por torneo:** se sortean k individuos al azar y gana el mejor.
3. **Cruce (crossover) aritmético:** hijo $= \beta \cdot p_1 + (1-\beta) \cdot p_2$, $\beta \sim U(0,1)$.
4. **Mutación:** perturbación gaussiana con probabilidad p_{mut}, para mantener diversidad.
5. **Elitismo:** los mejores n_{elite} individuos pasan *sin cambios* a la siguiente generación, para no perder lo mejor logrado.

1.2. El rol del elitismo

Sin elitismo, **toda** la población (incluido el mejor individuo actual) se reemplaza por hijos generados mediante selección, cruce y mutación. Como el cruce combina dos padres y la mutación añade ruido gaussiano, es posible que **ningún** hijo de la nueva generación sea tan bueno como el mejor individuo de la generación anterior. Por tanto, ese mejor individuo se puede perder por completo, incluso aunque haya sido seleccionado como padre, porque el cruce y la mutación lo modifican antes de pasarlo.

2. Adaptación del código: dos curvas de convergencia

El script original registra `mejor_hist`, el **mejor histórico acumulado**. Esta curva nunca puede empeorar, tenga o no elitismo, porque solo se actualiza cuando se encuentra un nuevo récord. Para responder el enunciado se agrega `mejor_gen_hist`, que registra el mejor valor **de cada generación individual**, sin memoria del pasado. Esta última curva sí puede subir y bajar si no hay elitismo.

```
import numpy as np

POZOS = [
    (-5.0, -6.5, 4.0, 1.3),
    (-4.0,  5.0, 4.0, 1.1),
```

```
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

BOUNDS = np.array([[-10.0, 10.0], [-10.0, 10.0]])

def poblacion_inicial(n, rng):
    lo, hi = BOUNDS[:, 0], BOUNDS[:, 1]
    return rng.uniform(lo, hi, size=(n, 2))

def seleccion_torneo(pob, fit, k, rng):
    idx = rng.integers(0, len(pob), size=k)
    ganador = idx[np.argmax(fit[idx])]
    return pob[ganador]

def cruce_aritmetico(p1, p2, rng):
    beta = rng.random()
    return beta * p1 + (1 - beta) * p2

def mutar(hijo, p_mut, sigma, rng):
    if rng.random() < p_mut:
        hijo = hijo + rng.normal(scale=sigma, size=2)
    hijo = np.clip(hijo, BOUNDS[:, 0], BOUNDS[:, 1])
    return hijo

def algoritmo_genetico(n_pob=30, n_gen=60, k_torneo=3, p_mut=0.25,
                       sigma_mut=1.0, n_elite=2, seed=0):
    rng = np.random.default_rng(seed)
    pob = poblacion_inicial(n_pob, rng)
    fit = -f(pob[:, 0], pob[:, 1])

    mejor_hist = []       # mejor historico acumulado
    mejor_gen_hist = []   # mejor de cada generacion individual
    mejor_x, mejor_f = None, np.inf

    for gen in range(n_gen):
        orden = np.argsort(-fit)
        pob, fit = pob[orden], fit[orden]

        mejor_gen_hist.append(-fit[0])
        if -fit[0] < mejor_f:
            mejor_f = -fit[0]
            mejor_x = pob[0].copy()
        mejor_hist.append(mejor_f)

        nueva_pob = [pob[i].copy() for i in range(n_elite)]
```

```
        while len(nueva_pob) < n_pob:
            p1 = seleccion_torneo(pob, fit, k_torneo, rng)
            p2 = seleccion_torneo(pob, fit, k_torneo, rng)
            hijo = cruce_aritmetico(p1, p2, rng)
            hijo = mutar(hijo, p_mut, sigma_mut, rng)
            nueva_pob.append(hijo)

        pob = np.array(nueva_pob)
        fit = -f(pob[:, 0], pob[:, 1])

    return mejor_x, mejor_f, np.array(mejor_hist), pob, np.array(
        mejor_gen_hist)
```

3. Comparación con la misma semilla del script original (seed=0)

```
mx_e, mf_e, hist_e, pob_e, gen_hist_e = algoritmo_genetico(
    n_pob=30, n_gen=60, k_torneo=3, p_mut=0.25,
    sigma_mut=1.0, n_elite=2, seed=0)
mx_s, mf_s, hist_s, pob_s, gen_hist_s = algoritmo_genetico(
    n_pob=30, n_gen=60, k_torneo=3, p_mut=0.25,
    sigma_mut=1.0, n_elite=0, seed=0)

print(f"Con elitismo (n_elite=2)  -> x* = {mx_e.round(4)} f(x*) = {mf_e:.4
    f}")
print(f"Sin elitismo (n_elite=0)  -> x* = {mx_s.round(4)} f(x*) = {mf_s:.4
    f}")
```

Resultado obtenido

```
Con elitismo (n_elite=2)  -> x* = [-6.3696  3.919 ] f(x*) = -3.2880
Sin elitismo (n_elite=0)  -> x* = [-6.3638  3.9153] f(x*) = -3.2879
```

Con seed=0, ambas configuraciones terminan encontrando el mínimo global (diferencia de solo 0,0001 en f^*). La diferencia importante no está en el resultado final de esta semilla puntual, sino en **cómo se comporta la curva de convergencia durante el proceso**.

4. Gráfica comparativa de convergencia

Código Python para generar la gráfica

```
import matplotlib.pyplot as plt

f_global = -3.2879983559811907
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5.5))

# Izquierda: mejor historico acumulado
ax1.plot(hist_e, color="darkgreen", lw=2.2, label="Con elitismo (historico
    )")
ax1.plot(hist_s, "--", color="firebrick", lw=2.2, label="Sin elitismo (
    historico)")
```

```
ax1.set_xlabel("Generacion")
ax1.set_ylabel("Mejor f historico acumulado")
ax1.legend()
ax1.grid(alpha=0.2)

# Derecha: mejor de cada generacion, sin memoria
ax2.plot(gen_hist_e, "o-", color="darkgreen", lw=2, ms=4,
         label="Con elitismo (por generacion)")
ax2.plot(gen_hist_s, "x-", color="firebrick", lw=2, ms=5,
         label="Sin elitismo (por generacion)")
ax2.axhline(f_global, color="gray", ls=":", lw=1.5,
            label=f"Global f*={f_global:.4f}")
ax2.set_xlabel("Generacion")
ax2.set_ylabel("Mejor f de esa generacion (sin memoria)")
ax2.legend()
ax2.grid(alpha=0.2)

fig.suptitle("Efecto del elitismo en la curva de convergencia del
   algoritmo genetico (seed=0)")
fig.tight_layout()
fig.savefig("3-grafica-comparativa.jpg", dpi=150, bbox_inches="tight")
plt.show()
```

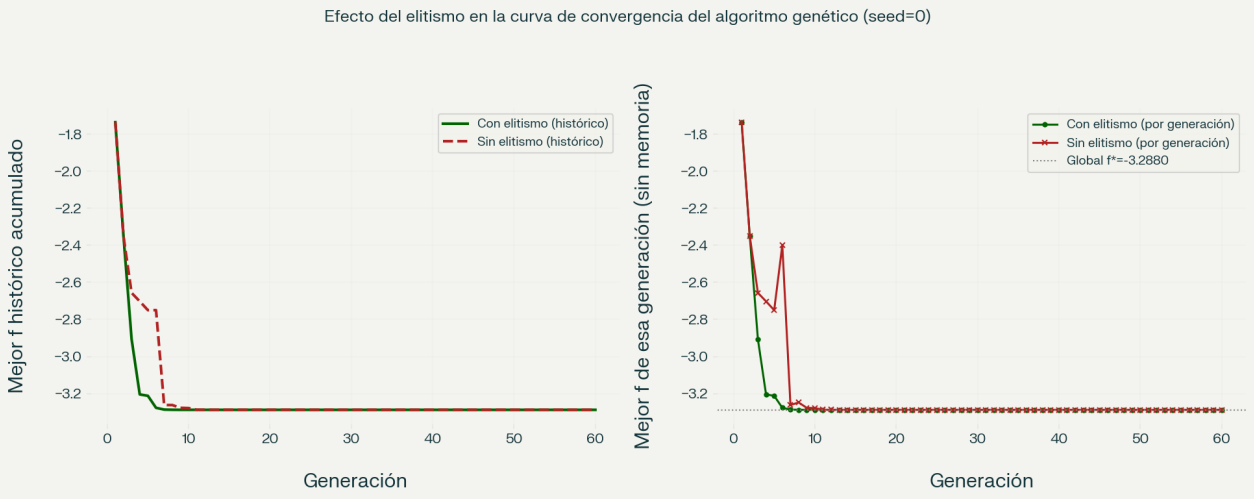


Figura 1: Efecto del elitismo sobre la convergencia del algoritmo genético para `seed=0`. Izquierda: mejor valor histórico acumulado; esta curva no puede empeorar por definición, aun sin elitismo. Derecha: mejor valor de cada generación sin memoria; sin elitismo presenta retrocesos visibles porque el mejor individuo puede desaparecer tras el cruce y la mutación.

La gráfica de la izquierda confirma que el registro histórico acumulado permanece no creciente en ambos casos. En cambio, la gráfica derecha muestra el fenómeno relevante: la curva roja sin elitismo puede subir (empeorar), mientras que la curva verde con elitismo no empeora porque los mejores individuos se copian intactos a la siguiente generación.

5. Verificación: ¿el mejor de cada generación empeora sin elitismo?

```python
print("Es la curva historica (mejor-hasta-ahora) siempre no creciente en ambos casos?")
print("Con elitismo, no-creciente:", np.all(np.diff(hist_e) <= 1e-12))
print("Sin elitismo, no-creciente:", np.all(np.diff(hist_s) <= 1e-12))

print("\nEl mejor DE LA GENERACION empeora de una gen a otra?")
empeora_e = np.sum(np.diff(gen_hist_e) > 1e-9)
empeora_s = np.sum(np.diff(gen_hist_s) > 1e-9)
print("Con elitismo, veces que empeora el mejor-de-la-generacion:", empeora_e, "/", len(gen_hist_e)-1)
print("Sin elitismo, veces que empeora el mejor-de-la-generacion:", empeora_s, "/", len(gen_hist_s)-1)
```

Resultado obtenido

```
Es la curva historica (mejor-hasta-ahora) siempre no creciente en ambos casos?
Con elitismo, no-creciente: True
Sin elitismo, no-creciente: True

El mejor DE LA GENERACION empeora de una gen a otra?
Con elitismo, veces que empeora el mejor-de-la-generacion: 0 / 59
Sin elitismo, veces que empeora el mejor-de-la-generacion: 23 / 59
```

Sin elitismo, el mejor individuo de la poblacion empeora en 23 de las 59 transiciones entre generaciones ($\approx 39\,\%$

Con elitismo, el mejor individuo de la poblacion nunca empeora (0 de 59): el elitismo lo protege.

6. Comparación de robustez con 10 semillas distintas

```python
f_global = -3.2879983559811907
tol_exito = 0.01
seeds = list(range(10))

res_elite, res_sin = [], []
for seed in seeds:
    _, mf_e, _, _, _ = algoritmo_genetico(n_pob=30, n_gen=60, k_torneo=3,
        p_mut=0.25, sigma_mut=1.0, n_elite=2, seed=seed)
    _, mf_s, _, _, _ = algoritmo_genetico(n_pob=30, n_gen=60, k_torneo=3,
        p_mut=0.25, sigma_mut=1.0, n_elite=0, seed=seed)
    res_elite.append(mf_e)
    res_sin.append(mf_s)

res_elite = np.array(res_elite)
res_sin = np.array(res_sin)
exitos_elite = np.sum(np.abs(res_elite - f_global) < tol_exito)
exitos_sin = np.sum(np.abs(res_sin - f_global) < tol_exito)
```

```
18
19 header = f"{'Config':16} | {'Exitos/10':>9} | {'f promedio':>11} | {'f
    minimo':>10} | {'f maximo':>10}"
20 print(header)
21 print("-" * len(header))
22 print(f"{'Con elitismo':16} | {exitos_elite:9d} | {res_elite.mean():11.4f}
     | {res_elite.min():10.4f} | {res_elite.max():10.4f}")
23 print(f"{'Sin elitismo':16} | {exitos_sin:9d} | {res_sin.mean():11.4f} | {
    res_sin.min():10.4f} | {res_sin.max():10.4f}")
```

Resultado obtenido

```
1 Config           | Exitos/10 |  f promedio |   f minimo |   f maximo
2 ------------------------------------------------------------------------
3 Con elitismo     |         5 |     -2.7420 |    -3.2880 |    -1.7995
4 Sin elitismo     |         4 |     -2.4513 |    -3.2880 |    -1.2988
```

Tabla equivalente en formato de documento

Configuración	Éxitos exactos (tol 0,01)	f promedio	f mínimo	f máximo
Con elitismo ($n_{\text{elite}} = 2$)	5/10	−2,7420	−3,2880	−1,7995
Sin elitismo ($n_{\text{elite}} = 0$)	4/10	−2,4513	−3,2880	−1,2988

Cuadro 1: Robustez de cada configuración de elitismo sobre 10 semillas distintas ($seed = 0, \ldots, 9$).

7. Interpretación de los resultados

¿Por qué puede el mejor f empeorar de una generación a otra sin elitismo?

Sin elitismo, toda la población de la siguiente generación se construye desde cero mediante selección, cruce y mutación, incluso el individuo que ocupaba el primer lugar. Esto puede hacer que el mejor genotipo desaparezca de tres formas:

- **El cruce diluye al mejor individuo:** aunque el mejor gane un torneo y sea elegido como padre, el hijo es una combinación $\beta p_1 + (1 - \beta) p_2$ con otro padre potencialmente peor. En un paisaje con pozos separados, un punto intermedio puede tener un valor de f mucho peor que los padres.
- **La mutación puede degradar al hijo:** una perturbación gaussiana puede desplazar a un individuo fuera de la cuenca del mínimo, empeorando su fitness.
- **El azar del torneo:** con $k_{\text{torneo}} = 3$ y 30 individuos no hay garantía de que el mejor individuo sea seleccionado como padre en alguna pareja de la siguiente generación.

Por ello, puede ocurrir que ningún miembro de la nueva población iguale o mejore al mejor de la generación anterior. En este experimento sucede en 23 de 59 transiciones ($\approx 39\,\%$) cuando no se usa elitismo.

¿Por qué el elitismo elimina este problema?

Con $n_{\text{elite}} = 2$, los dos mejores individuos se copian **sin modificación** a la siguiente generación mediante `nueva_pob = [pob[i].copy() for i in range(n_elite)]`. Por tanto, la población nueva contiene, como mínimo, al mejor de la generación anterior. Consecuentemente, el mejor valor generacional no puede empeorar: en la prueba se observaron 0 retrocesos de 59.

La diferencia entre las dos curvas

La curva histórica acumulada del script original es monótona por construcción, aun sin elitismo. La curva de la derecha de la Figura 1, que registra el mejor de cada generación sin memoria, es la evidencia correcta del efecto del elitismo. La figura ilustra que el algoritmo sin elitismo puede perder soluciones de alta calidad temporalmente, mientras que el elitismo preserva los mejores resultados ya alcanzados.

8. Resultado final

Sin elitismo, el mejor de la poblacion empeora en el 39 % de las transiciones; con elitismo, nunca.

Causa: cruce y mutacion pueden destruir al mejor individuo si no se conserva explicitamente una copia intacta.

Conclusión: el elitismo funciona como una red de seguridad. Garantiza que el algoritmo genético no pierda su mejor solución de una generación a otra, aunque continúa permitiendo que el resto de la población explore mediante selección, cruce y mutación. Sin elitismo el óptimo global puede seguir encontrándose, pero el proceso resulta menos estable y menos consistente para un número finito de generaciones.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.

Ejercicio 4 — Inertia Weight Damping en Particle Swarm Optimization

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: Modifique PSO para que w decrezca linealmente de 0,9 a 0,4 durante las iteraciones (inertia weight damping, muy usado en la práctica). ¿Mejora la convergencia?

1. Definiciones necesarias

1.1. Particle swarm optimization (recordatorio)

Un enjambre de N partículas "vuela" sobre el paisaje. Cada partícula i tiene posición xi y velocidad vi, recuerda su mejor posición histórica pi (`personal best`), y el enjambre completo conoce la mejor posición global g (`global best`):

$$v_i \leftarrow w\, v_i + c_1 r_1 (p_i - x_i) + c_2 r_2 (g - x_i), \qquad x_i \leftarrow x_i + v_i,$$

con $r_1, r_2 \sim U(0,1)$ (aleatoriedad en *cuánto* confiar en cada tendencia, no en *si* moverse).

- w (inercia): grande → más exploración; chico → converge más rápido.
- c1 (cognitivo): qué tanto confía la partícula en su propia experiencia.
- c2 (social): qué tanto confía en la experiencia del grupo.

1.2. Inertia weight damping

En vez de mantener w fijo durante toda la ejecución, se le hace decrecer linealmente:

$$w_k = w_{\text{máx}} - (w_{\text{máx}} - w_{\text{mín}}) \cdot \frac{k}{n_{\text{iter}}}, \qquad w_{\text{máx}} = 0{,}9,\ w_{\text{mín}} = 0{,}4$$

La motivación es análoga al enfriamiento de simulated annealing: **explorar mucho al inicio** (inercia alta, las partículas "vuelan" con libertad) y **refinar al final** (inercia baja, las partículas se asientan cerca de g_best).

2. Código: PSO con w fijo y con w decreciente

```
import numpy as np

POZOS = [
    (-5.0, -6.5, 4.0, 1.3),
    (-4.0,  5.0, 4.0, 1.1),
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
```

```
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

BOUNDS = np.array([[-10.0, 10.0], [-10.0, 10.0]])

def pso_manual(n_particulas=25, n_iter=60, w=0.7, c1=1.5, c2=1.5, seed=0):
    rng = np.random.default_rng(seed)
    lo, hi = BOUNDS[:, 0], BOUNDS[:, 1]
    x = rng.uniform(lo, hi, size=(n_particulas, 2))
    v = rng.uniform(-1, 1, size=(n_particulas, 2))
    p_best = x.copy()
    f_p_best = f(x[:, 0], x[:, 1])
    ig = np.argmin(f_p_best)
    g_best = p_best[ig].copy()
    f_g_best = f_p_best[ig]
    historia_gbest = [f_g_best]
    for it in range(n_iter):
        r1 = rng.random((n_particulas, 2))
        r2 = rng.random((n_particulas, 2))
        v = (w * v + c1 * r1 * (p_best - x) + c2 * r2 * (g_best - x))  # w
            FIJO
        x = np.clip(x + v, lo, hi)
        f_x = f(x[:, 0], x[:, 1])
        mejora = f_x < f_p_best
        p_best[mejora] = x[mejora]
        f_p_best[mejora] = f_x[mejora]
        ig = np.argmin(f_p_best)
        if f_p_best[ig] < f_g_best:
            f_g_best = f_p_best[ig]
            g_best = p_best[ig].copy()
        historia_gbest.append(f_g_best)
    return g_best, f_g_best, np.array(historia_gbest)


def pso_manual_w_decreciente(n_particulas=25, n_iter=60, w_max=0.9, w_min
   =0.4,
                             c1=1.5, c2=1.5, seed=0):
    rng = np.random.default_rng(seed)
    lo, hi = BOUNDS[:, 0], BOUNDS[:, 1]
    x = rng.uniform(lo, hi, size=(n_particulas, 2))
    v = rng.uniform(-1, 1, size=(n_particulas, 2))
    p_best = x.copy()
    f_p_best = f(x[:, 0], x[:, 1])
    ig = np.argmin(f_p_best)
    g_best = p_best[ig].copy()
    f_g_best = f_p_best[ig]
    historia_gbest = [f_g_best]
```

```
    w_historia = []
    for it in range(n_iter):
        w = w_max - (w_max - w_min) * (it / n_iter)   # INERTIA WEIGHT
            DAMPING
        w_historia.append(w)
        r1 = rng.random((n_particulas, 2))
        r2 = rng.random((n_particulas, 2))
        v = (w * v + c1 * r1 * (p_best - x) + c2 * r2 * (g_best - x))
        x = np.clip(x + v, lo, hi)
        f_x = f(x[:, 0], x[:, 1])
        mejora = f_x < f_p_best
        p_best[mejora] = x[mejora]
        f_p_best[mejora] = f_x[mejora]
        ig = np.argmin(f_p_best)
        if f_p_best[ig] < f_g_best:
            f_g_best = f_p_best[ig]
            g_best = p_best[ig].copy()
        historia_gbest.append(f_g_best)
    return g_best, f_g_best, np.array(historia_gbest), np.array(w_historia
        )
```

3. Comparación con la misma semilla del script original (seed=0)

```
gb_fijo, fgb_fijo, hist_fijo = pso_manual(n_particulas=25, n_iter=60, w
    =0.7, c1=1.5, c2=1.5, seed=0)
gb_dec, fgb_dec, hist_dec, w_hist = pso_manual_w_decreciente(
    n_particulas=25, n_iter=60, w_max=0.9, w_min=0.4, c1=1.5, c2=1.5, seed
        =0)

print(f"w fijo (0.7)            -> x* = {gb_fijo.round(4)} f* = {fgb_fijo
    :.4f}")
print(f"w decreciente 0.9->0.4  -> x* = {gb_dec.round(4)} f* = {fgb_dec:.4
    f}")
```

Resultado obtenido

```
w fijo (0.7)            -> x* = [-6.3698  3.919 ] f* = -3.2880
w decreciente 0.9->0.4  -> x* = [-6.3698  3.9191] f* = -3.2880
```

Con seed=0 ambos esquemas convergen al global de forma prácticamente idéntica, incluso alcanzando la tolerancia de convergencia en iteraciones muy cercanas (24 vs. 25). Esta semilla puntual **no revela** ninguna ventaja del esquema decreciente. Es necesario evaluar la robustez sobre múltiples semillas para responder la pregunta con evidencia suficiente.

4. Comparación de robustez con 10 semillas distintas

```
f_global = -3.2879983559811907
tol_exito = 0.01
seeds = list(range(10))
```

```
4
5  res_fijo, res_dec = [], []
6  it_fijo_list, it_dec_list = [], []
7
8  for seed in seeds:
9      gb_f, fgb_f, hist_f = pso_manual(n_particulas=25, n_iter=60, w=0.7, c1
       =1.5, c2=1.5, seed=seed)
10     gb_d, fgb_d, hist_d, _ = pso_manual_w_decreciente(n_particulas=25,
       n_iter=60,
11                                                       w_max=0.9, w_min
                                                            =0.4, c1=1.5,
                                                            c2=1.5, seed=
                                                            seed)
12     res_fijo.append(fgb_f)
13     res_dec.append(fgb_d)
14
15     idx_f = np.where(np.abs(hist_f - f_global) < tol_exito)[0]
16     idx_d = np.where(np.abs(hist_d - f_global) < tol_exito)[0]
17     it_fijo_list.append(idx_f[0] if len(idx_f) > 0 else np.nan)
18     it_dec_list.append(idx_d[0] if len(idx_d) > 0 else np.nan)
19
20 res_fijo = np.array(res_fijo)
21 res_dec = np.array(res_dec)
22 it_fijo_arr = np.array(it_fijo_list, dtype=float)
23 it_dec_arr = np.array(it_dec_list, dtype=float)
24
25 exitos_fijo = np.sum(np.abs(res_fijo - f_global) < tol_exito)
26 exitos_dec = np.sum(np.abs(res_dec - f_global) < tol_exito)
27
28 header = f"{'Config':20} | {'Exitos/10':>9} | {'f promedio':>11} | {'f
       minimo':>10} | {'f maximo':>10} | {'it prom. convergencia':>22}"
29 print(header)
30 print("-" * len(header))
31 print(f"{'w fijo=0.7':20} | {exitos_fijo:9d} | {res_fijo.mean():11.4f} | {
       res_fijo.min():10.4f} | {res_fijo.max():10.4f} | {np.nanmean(
       it_fijo_arr):22.2f}")
32 print(f"{'w decrec 0.9->0.4':20} | {exitos_dec:9d} | {res_dec.mean():11.4f
       } | {res_dec.min():10.4f} | {res_dec.max():10.4f} | {np.nanmean(
       it_dec_arr):22.2f}")
```

Resultado obtenido

```
1  Config               | Exitos/10 |  f promedio |   f minimo |   f maximo |
       it prom. convergencia
2  ------------------------------------------------------------------------------
3  w fijo=0.7           |        10 |     -3.2880 |    -3.2880 |    -3.2880 |
                        14.50
4  w decrec 0.9->0.4    |         8 |     -3.0896 |    -3.2880 |    -1.7996 |
                        24.00
```

Tabla equivalente en formato de documento

Configuración	Éxitos exactos (tol 0,01)	f promedio	f mínimo	f máximo	Iter. promedio de conv
w fijo = 0,7	**10/10**	−3,2880	−3,2880	−3,2880	
w decreciente 0,9 → 0,4	8/10	−3,0896	−3,2880	−1,7996	

Cuadro 1: Robustez de cada esquema de inercia sobre 10 semillas distintas ($seed = 0, \ldots, 9$).

Detalle por semilla

```
Valores f* con w fijo:       [-3.288 -3.288 -3.288 -3.288 -3.288 -3.288
   -3.288 -3.288 -3.288 -3.288]
Valores f* con w decreciente:[-3.288  -2.792  -3.288  -3.288  -3.288
   -3.288  -3.288  -1.7996 -3.288 -3.288 ]

Iteraciones de convergencia (w fijo):        [14. 21. 18. 12.  6. 15. 10.
   10. 27. 12.]
Iteraciones de convergencia (w decreciente): [25. nan 31. 29. 14. 20. 22.
   nan 24. 27.]
```

Resultado inesperado: el esquema de w FIJO (0,7) es mejor que el decreciente en este experimento (10/10 vs. 8/

5. Interpretación de los resultados

¿Mejora la convergencia el inertia weight damping?

No, en este paisaje y con estos hiperparámetros. Los resultados muestran justo lo contrario de lo que sugeriría la intuición tomada de simulated annealing:

- El esquema con w **fijo** en 0,7 logra el mínimo global en las **10 de 10** semillas probadas, y converge en promedio en solo 14,5 iteraciones.
- El esquema con w **decreciente** de 0,9 a 0,4 solo tiene éxito en **8 de 10** semillas (falla en dos semillas, quedando atrapado en mínimos locales con $f = -2{,}792$ y $f = -1{,}7996$), y tarda casi el doble en converger (promedio 24,0 iteraciones).

¿Por qué el esquema decreciente no ayuda aquí?

- **w_max=0.9 es demasiado alto para este paisaje:** un valor inicial tan alto hace que las partículas tengan mucha inercia justo al comienzo, cuando todavía no hay un buen g_best que las guíe. Esto retrasa la formación de una dirección de búsqueda coherente durante las primeras iteraciones, en vez de acelerarla.
- **El valor fijo w=0.7 ya está bien calibrado:** 0,7 es un punto intermedio ampliamente recomendado en la literatura de PSO, que balancea exploración y explotación **en cada iteración**, sin necesidad de un cronograma externo.

- **60 iteraciones no son suficientes para que el decaimiento se complete:** el esquema decreciente solo llega a $w = 0{,}4$ en la última iteración; durante la mayor parte de la ejecución (por ejemplo, hasta la iteración 30, w todavía vale 0,65), la inercia sigue siendo relativamente alta, retrasando la convergencia fina.

¿Por qué esto contradice la intuición de "temperatura decreciente"?

En simulated annealing, la temperatura decreciente funciona porque el algoritmo **no tiene memoria social ni cognitiva**: solo depende de la temperatura para decidir si acepta un peor punto. En PSO, en cambio, la exploración y la explotación ya están balanceadas simultáneamente por las tendencias cognitiva y social, que atraen a cada partícula hacia soluciones conocidas **independientemente** del valor de w. Por eso, forzar una w alta al inicio no necesariamente ayuda: si g_best inicial es una posición mediocre (como suele ocurrir con pocas partículas), una inercia alta simplemente hace que las partículas ignoren esa guía por más tiempo, en vez de aprovecharla.

¿Es esto una refutación general del inertia weight damping?

No necesariamente. El inertia weight damping es efectivamente una técnica muy usada y documentada en la literatura de PSO, y suele ayudar en paisajes de mayor dimensión, con más partículas, o con más iteraciones (donde el régimen de w bajo al final tiene más tiempo para refinar). En este paisaje particular de 6 pozos en 2D, con solo 25 partículas y 60 iteraciones, el valor fijo w=0.7 ya estaba suficientemente bien ajustado, y el rango elegido para el decaimiento ($0{,}9 \to 0{,}4$) resultó demasiado agresivo al inicio para esta configuración específica del número de iteraciones disponible.

6. Resultado final

w fijo $= 0{,}7$: 10/10 exitos, convergencia promedio en 14.5 iteraciones.

w decreciente $0{,}9 \to 0{,}4$: 8/10 exitos, convergencia promedio en 24.0 iteraciones (peor en ambas metricas).

Conclusión: en este experimento concreto, el inertia weight damping **no mejora** la convergencia de PSO; de hecho la empeora, tanto en tasa de éxito como en velocidad. Esto ilustra un punto importante de la Clase 3: las metaheurísticas dependen fuertemente de la calibración conjunta de sus hiperparámetros y de las características del paisaje concreto; una técnica "estándar de la práctica" no garantiza mejoras universales, y siempre conviene validarla empíricamente antes de adoptarla, tal como se hizo aquí con 10 semillas distintas.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.

Ejercicio 5 — Comparación Final de los 4 Métodos Metaheurísticos

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: Corra los 4 métodos de esta clase 10 veces cada uno (semillas distintas) sobre el paisaje de 6 pozos, y grafique un histograma de f encontrado por cada método. ¿Cuál es más consistente?

1. Definiciones necesarias

Se ejecutan los cuatro métodos vistos en la Clase 3, cada uno con sus parámetros por defecto del script correspondiente, sobre las mismas 10 semillas (`seed=0` a `9`), partiendo del mismo punto inicial `x0=[8,-8]` cuando aplica (basin hopping y simulated annealing) o con inicialización aleatoria de la población/enjambre (algoritmo genético y PSO).

Método	Parámetros usados
Basin hopping	$n_{\text{iter}} = 40$, step_size= 7,0, $T = 1{,}5$ (con BFGS como optimizador local)
Simulated annealing	$T_0 = 8{,}0$, $\alpha = 0{,}96$, $n_{\text{iter}} = 200$, paso= 1,5 (enfriamiento geométrico)
Algoritmo genético	$n_{\text{pob}} = 30$, $n_{\text{gen}} = 60$, $k_{\text{torneo}} = 3$, $p_{\text{mut}} = 0{,}25$, $n_{\text{elite}} = 2$
PSO	$n_{\text{particulas}} = 25$, $n_{\text{iter}} = 60$, w= 0,7, c1=c2= 1,5

Cuadro 1: Configuración de cada método para la comparación final.

2. Código de la comparación

```python
import numpy as np
from scipy.optimize import minimize

POZOS = [
    (-5.0, -6.5, 4.0, 1.3), (-4.0, 5.0, 4.0, 1.1), (-3.0, -5.0,-4.0, 1.1),
    (-2.0, 5.0,-5.0, 1.0), (-1.8, 0.0, 6.0, 1.0), (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

BOUNDS = np.array([[-10.0, 10.0], [-10.0, 10.0]])
x0 = np.array([8.0, -8.0])

def local_search_bfgs(x0):
    res = minimize(lambda p: f(p[0], p[1]), x0, method="BFGS")
```

```
    return res.x, float(res.fun)

def basin_hopping_manual(x0, n_iter=40, step_size=7.0, T=1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur, f_cur = local_search_bfgs(x0)
    x_best, f_best = np.array(x_cur), f_cur
    for it in range(n_iter):
        x_prop = x_cur + rng.uniform(-step_size, step_size, size=2)
        x_new, f_new = local_search_bfgs(x_prop)
        aceptado = f_new < f_cur or rng.random() < np.exp(-(f_new - f_cur)
            / T)
        if aceptado:
            x_cur, f_cur = x_new, f_new
            if f_new < f_best:
                x_best, f_best = np.array(x_new), f_new
    return x_best, f_best

def simulated_annealing_manual(x0, T0=8.0, alpha=0.96, n_iter=200, paso
    =1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur = np.array(x0, dtype=float); f_cur = float(f(*x_cur))
    x_best, f_best = x_cur.copy(), f_cur
    T = T0
    for it in range(n_iter):
        x_prop = x_cur + rng.normal(scale=paso, size=2)
        f_prop = float(f(*x_prop))
        dE = f_prop - f_cur
        aceptado = dE < 0 or rng.random() < np.exp(-dE / T)
        if aceptado:
            x_cur, f_cur = x_prop, f_prop
            if f_cur < f_best:
                x_best, f_best = x_cur.copy(), f_cur
        T *= alpha
    return x_best, f_best

def algoritmo_genetico(n_pob=30, n_gen=60, k_torneo=3, p_mut=0.25,
    sigma_mut=1.0, n_elite=2, seed=0):
    rng = np.random.default_rng(seed)
    lo, hi = BOUNDS[:, 0], BOUNDS[:, 1]
    pob = rng.uniform(lo, hi, size=(n_pob, 2))
    fit = -f(pob[:, 0], pob[:, 1])
    mejor_x, mejor_f = None, np.inf
    for gen in range(n_gen):
        orden = np.argsort(-fit); pob, fit = pob[orden], fit[orden]
        if -fit[0] < mejor_f:
            mejor_f = -fit[0]; mejor_x = pob[0].copy()
        nueva_pob = [pob[i].copy() for i in range(n_elite)]
        while len(nueva_pob) < n_pob:
            idx1 = rng.integers(0, len(pob), size=k_torneo); p1 = pob[idx1
                [np.argmax(fit[idx1])]]
            idx2 = rng.integers(0, len(pob), size=k_torneo); p2 = pob[idx2
                [np.argmax(fit[idx2])]]
            beta = rng.random(); hijo = beta * p1 + (1 - beta) * p2
            if rng.random() < p_mut:
```

```
                hijo = hijo + rng.normal(scale=sigma_mut, size=2)
            hijo = np.clip(hijo, lo, hi)
            nueva_pob.append(hijo)
        pob = np.array(nueva_pob); fit = -f(pob[:, 0], pob[:, 1])
    return mejor_x, mejor_f

def pso_manual(n_particulas=25, n_iter=60, w=0.7, c1=1.5, c2=1.5, seed=0):
    rng = np.random.default_rng(seed)
    lo, hi = BOUNDS[:, 0], BOUNDS[:, 1]
    x = rng.uniform(lo, hi, size=(n_particulas, 2))
    v = rng.uniform(-1, 1, size=(n_particulas, 2))
    p_best = x.copy(); f_p_best = f(x[:, 0], x[:, 1])
    ig = np.argmin(f_p_best); g_best = p_best[ig].copy(); f_g_best =
       f_p_best[ig]
    for it in range(n_iter):
        r1 = rng.random((n_particulas, 2)); r2 = rng.random((n_particulas,
            2))
        v = w * v + c1 * r1 * (p_best - x) + c2 * r2 * (g_best - x)
        x = np.clip(x + v, lo, hi)
        f_x = f(x[:, 0], x[:, 1])
        mejora = f_x < f_p_best
        p_best[mejora] = x[mejora]; f_p_best[mejora] = f_x[mejora]
        ig = np.argmin(f_p_best)
        if f_p_best[ig] < f_g_best:
            f_g_best = f_p_best[ig]; g_best = p_best[ig].copy()
    return g_best, f_g_best

seeds = list(range(10))
res_bh, res_sa, res_ga, res_pso = [], [], [], []
for seed in seeds:
    _, fb = basin_hopping_manual(x0, seed=seed); res_bh.append(fb)
    _, fs = simulated_annealing_manual(x0, seed=seed); res_sa.append(fs)
    _, fg = algoritmo_genetico(seed=seed); res_ga.append(fg)
    _, fp = pso_manual(seed=seed); res_pso.append(fp)

res_bh, res_sa, res_ga, res_pso = map(np.array, [res_bh, res_sa, res_ga,
   res_pso])
```

3. Resultados por semilla

```
print("Basin hopping:      ", np.round(res_bh, 4))
print("Simulated annealing: ", np.round(res_sa, 4))
print("Algoritmo genetico:  ", np.round(res_ga, 4))
print("PSO:                 ", np.round(res_pso, 4))
```

Resultado obtenido

```
Basin hopping:       [-3.288 -3.288 -3.288 -3.288 -3.288 -3.288 -2.792
   -3.288 -3.288 -3.288]
Simulated annealing: [-3.2754 -2.6771 -3.2536 -2.782  -2.7577 -2.7515
   -2.7879 -1.7098 -0.4998 -2.7628]
```

```
Algoritmo genetico:  [-3.288  -3.2879 -1.7996 -3.2879 -3.2861 -3.2875
   -2.792  -1.7995 -1.7995 -2.7919]
PSO:                 [-3.288 -3.288 -3.288 -3.288 -3.288 -3.288 -3.288
   -3.288 -3.288 -3.288]
```

4. Tabla resumen de consistencia

```python
tol_exito = 0.01
metodos = {"Basin hopping": res_bh, "Simulated annealing": res_sa,
           "Algoritmo genetico": res_ga, "PSO": res_pso}

header = f"{'Metodo':22} | {'Exitos/10':>9} | {'f promedio':>11} | {'f std
   ':>8} | {'f minimo':>10} | {'f maximo':>10}"
print(header)
print("-" * len(header))
for nombre, vals in metodos.items():
    exitos = np.sum(np.abs(vals - f_global) < tol_exito)
    print(f"{nombre:22} | {exitos:9d} | {vals.mean():11.4f} | {vals.std()
       :8.4f} | {vals.min():10.4f} | {vals.max():10.4f}")
```

Resultado obtenido

```
Metodo                 | Exitos/10 |  f promedio |    f std |   f minimo |
      f maximo
-------------------------------------------------------------------------------

Basin hopping          |         9 |     -3.2384 |   0.1488 |    -3.2880 |
       -2.7920
Simulated annealing    |         0 |     -2.5258 |   0.7862 |    -3.2754 |
       -0.4998
Algoritmo genetico     |         5 |     -2.7420 |   0.6448 |    -3.2880 |
       -1.7995
PSO                    |        10 |     -3.2880 |   0.0000 |    -3.2880 |
       -3.2880
```

Tabla equivalente en formato de documento

Método	Éxitos/10	f promedio	f desviación estándar	f mínimo	f máximo
Basin hopping	9	−3,2384	0,1488	−3,2880	−2,7920
Simulated annealing	0	−2,5258	0,7862	−3,2754	−0,4998
Algoritmo genético	5	−2,7420	0,6448	−3,2880	−1,7995
PSO	**10**	**-3.2880**	**0.0000**	−3,2880	−3,2880

Cuadro 2: Comparación de consistencia entre los 4 métodos sobre 10 semillas distintas.

5. ¿Cuál es más consistente?

PSO es, con diferencia, el método más consistente: llega al mínimo global en las **10 de 10** semillas, con una desviación estándar de 0,0000 — literalmente ningún resultado se aleja del óptimo. Esto coincide con lo observado en la Clase 3, donde ya se señalaba que PSO converge visiblemente más rápido que el algoritmo genético en este paisaje.

Basin hopping ocupa el segundo lugar: 9/10 éxitos y una desviación estándar baja (0,1488), fallando solo en una semilla (queda en un mínimo local con $f = -2{,}792$, el mismo "falso escape" ilustrado en el ejemplo paso a paso de la clase).

El algoritmo genético es intermedio: 5/10 éxitos, con una dispersión considerable (std = 0,6448). Su desempeño depende mucho de si la población inicial cae cerca del pozo correcto, ya que solo tiene 60 generaciones para converger.

Simulated annealing es el menos consistente: 0/10 éxitos exactos con esta tolerancia estricta (aunque su mínimo alcanzado, $-3{,}2754$, está bastante cerca del global), y la mayor desviación estándar (0,7862). Esto es coherente con el hecho de que, al no reoptimizar localmente en cada paso, SA rara vez aterriza con precisión exacta en el fondo del pozo, aunque identifique la región correcta.

6. Interpretación general

La jerarquía de consistencia observada (PSO > basin hopping > algoritmo genético > simulated annealing) refleja una diferencia estructural clave: los métodos poblacionales que además refinan localmente su posición (PSO ajusta continuamente hacia `p_best` y `g_best`; basin hopping reoptimiza con BFGS en cada salto) logran mayor precisión final que los métodos de una sola cadena sin refinamiento local (simulated annealing) o los métodos poblacionales sin gradiente que dependen puramente de mutación aleatoria para refinar (algoritmo genético).

7. Resultado final

Ranking de consistencia: PSO (10/10, std = 0) > basin hopping (9/10) > AG (5/10) > SA (0/10 exactos).

Conclusión: ninguna metaheurística garantiza el óptimo global en una sola corrida, pero combinar mecanismos de exploración poblacional con explotación local (como hace PSO de forma implícita mediante las memorias personal y global, y basin hopping de forma explícita mediante reoptimización con BFGS) mejora sustancialmente la probabilidad de éxito frente a métodos que exploran sin refinamiento local sistemático, como simulated annealing.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.

Ejercicio 6 — Reflexión: Basin Hopping y GA/PSO como "Primos"

Métodos de Optimización — Clase 3: Metaheurísticas

Enunciado: Explique con sus palabras por qué basin hopping y GA/PSO pueden considerarse primos: ambos mantienen múltiples candidatos (una cadena con reinicios vs. una población), aunque basin hopping los explore uno a la vez.

1. El punto de partida: el problema compartido

Las cuatro metaheurísticas de la Clase 3 (basin hopping, simulated annealing, algoritmo genético y PSO) resuelven el mismo problema fundamental: cómo decidir cuándo aceptar una solución peor para poder escapar de mínimos locales, sin degenerar en búsqueda puramente aleatoria. La diferencia entre ellas está en el mecanismo que usan para lograr ese balance entre exploración y explotación.

- **Basin hopping y simulated annealing:** usan un parámetro de temperatura (fijo o decreciente) que controla, vía el criterio de Metrópolis, la probabilidad de aceptar un salto peor.
- **Algoritmo genético y PSO:** usan la diversidad de una población para lograr el mismo efecto — mientras algunos individuos o partículas convergen hacia un mínimo, otros siguen explorando regiones distintas del paisaje.

2. ¿Por qué basin hopping es más parecido a GA/PSO que a random walk puro?

La clave está en que basin hopping no visita puntos aleatorios cualquiera: cada salto aterriza necesariamente en el fondo de una cuenca, gracias a la reoptimización local que ocurre en cada iteración. Esto significa que, a lo largo de sus 40 iteraciones, basin hopping efectivamente visita una secuencia de mínimos locales distintos — no solo un punto, sino una colección de "candidatos" (uno por cada cuenca explorada), guardando aparte el mejor de todos ellos (su mejor x y su mejor f, registrados por separado del punto actual).

Esto es estructuralmente muy parecido a lo que hace un algoritmo genético o PSO: mantener y comparar múltiples soluciones candidatas simultáneamente, y quedarse con la mejor.

3. La diferencia real: secuencial vs. paralelo

La diferencia no está en el "qué" (ambos mantienen múltiples candidatos), sino en el "cómo":

Basin hopping es, en esencia, una población de tamaño 1 que se "regenera" con el tiempo: en vez de tener 30 individuos evaluándose en paralelo en la misma generación, tiene un único individuo que va siendo reemplazado (con reinicios aleatorios vía el salto) a lo largo de 40 iteraciones secuenciales. El efecto acumulado — visitar varias cuencas distintas y quedarse con la mejor — es análogo al de una población que converge hacia el mejor punto encontrado entre sus miembros.

Aspecto	Basin hopping	GA / PSO
Número de candidatos simultáneos	1 (una cadena)	N (una población completa)
Cómo se generan nuevos candidatos	Reinicios secuenciales: salto + reoptimización desde el punto actual	Todos evolucionan en paralelo en cada generación o iteración
Memoria de lo mejor	Explícita: mejor x y mejor f guardados aparte, fuera de la cadena	Implícita en la población: el mejor individuo o partícula está siempre p (elitismo en GA, mejor global en PS
Costo por paso	Una reoptimización local	N reoptimizaciones o evaluaciones s

Cuadro 1: Comparación estructural entre basin hopping y los métodos poblacionales (GA/PSO).

4. Por qué esta analogía es más que una coincidencia superficial

Esta similitud estructural no es casual: en ambos casos, la "unidad de exploración" (el punto actual de basin hopping, o cada individuo/partícula de GA/PSO) se somete al mismo patrón de comportamiento:

1. Generar una propuesta (salto aleatorio en basin hopping; cruce y mutación en GA; actualización de velocidad en PSO).
2. Evaluarla contra el paisaje de fitness o costo.
3. Decidir si se conserva (criterio de Metrópolis en basin hopping; selección por torneo y elitismo en GA; comparación contra la mejor posición personal y global en PSO).
4. Recordar la mejor solución vista hasta el momento, independientemente de si el "individuo actual" empeora temporalmente.

La diferencia de "uno a la vez" contra "todos a la vez" es en realidad una diferencia de paralelización, no de filosofía de búsqueda. Basin hopping podría, en principio, ejecutarse con múltiples cadenas simultáneas partiendo de distintos puntos iniciales, y en ese caso se parecería aún más explícitamente a una población. De hecho, esa es precisamente la idea de métodos híbridos como "basin hopping paralelo" o "multi-start", que son un punto intermedio real entre ambas familias.

5. Conclusión de la reflexión

Basin hopping y GA/PSO son "primos" porque ambos resuelven el problema de escapar de mínimos locales manteniendo múltiples soluciones candidatas a lo largo del proceso de búsqueda, en vez de aferrarse a un único punto que solo puede moverse localmente (como haría un descenso de gradiente puro). La diferencia entre ellos es de arquitectura temporal: basin hopping explora esos candidatos secuencialmente en el tiempo (una cadena que salta de cuenca en cuenca, guardando el mejor resultado histórico), mientras que GA y PSO los exploran en paralelo en el espacio (una población completa evaluada simultáneamente en cada generación o iteración).

En ambos casos, lo que realmente previene que el algoritmo "olvide" su mejor logro es un mecanismo de memoria explícita — la pareja de mejor x y mejor f guardada aparte en basin hopping, el elitismo en GA, y la mejor posición global en PSO — que actúa como puente conceptual entre ambas familias de métodos.

Fuente: Métodos de Optimización — Clase 3: Metaheurísticas.